[인계] 이벤트 로그 / 백엔드 연동 가이드 (초보·AI 사용 가능)

작성일: 2026-10-08
저장소: https://github.com/Sang-Hyeop/K_newdeal_visionAI
코드 기준: **main 최신** (PR #8 머지 완료)

---

0. 한 줄 요약

감지팀이 만든 **이벤트 JSONL 파일**을 읽어서 DB에 넣고, 대시보드가 쓸 **조회 API**를 만들면 됩니다.

- YOLO / 모델 학습은 하지 않습니다.
- `.pt` 없어도 샘플 JSON만으로 시작 가능합니다.

```
git clone https://github.com/Sang-Hyeop/K_newdeal_visionAI.git
cd K_newdeal_visionAI
git switch main
git pull --ff-only
```

이미 clone 했다면:

```
git fetch origin
git switch main
git pull --ff-only
```

---

1. AI(Cursor/Codex 등)로 작업하는 방법

1.1 추천 사용법

1. 저장소를 clone한 뒤, AI 채팅에 **아래 문서들을 첨부**하고 요청한다.
2. AI에게 시킬 일 예시(복붙용):

```
너는 백엔드 담당이다. YOLO/학습은 하지 마라.
docs/team-integration-pack-2026-10-07.md 와
```

```
docs/team-samples/2026-10-07/ 샘플을 기준으로
1) events_v1.jsonl / observations_v1.jsonl 파서
2) DB 테이블(event_id UNIQUE, severity NULL 허용, evidence JSON 원문 보존)
3) API 3개: 이벤트 목록/상세, feature_status 최신
를 만들어라.
규칙: unconfirmed면 severity=null, 빈 관측을 SAFE로 만들지 말 것, distance_meters는 null.
샘플만으로 동작하게 하고 테스트 코드를 추가해라.
```

1.2 AI에게 시키면 안 되는 것

- 모델 학습 / `.pt` 다운로드 강제
- `distance_meters`를 임의로 채워 m 단위 표시
- “이벤트 없음 = 현장 전체 안전(SAFE)” 처리
- 구역 CRITICAL을 체류 SAFE로 덮어쓰기

1.3 AI에게 꼭 첨부할 파일

우선순위	경로	이유
필수	`docs/team-integration-pack-2026-10-07.md`	메인 연동 안내·체크리스트
필수	`docs/team-samples/2026-10-07/` 전체	실제 JSON 샘플
필수	`docs/roi-wording-2026-10-07.md`	SAFE/ROI 말투(API 문서·UI 문구)
권장	`docs/detection-output-contract.md`	스키마 상세
권장	`docs/team-handoff-index-2026-10-07.md`	문서 인덱스
참고	`docs/event-module-v1.md`	실행기 설명(백엔드 필수는 아님)
참고	`docs/demo-feature-show-plan-2026-10-07.md`	영상 1~7이 무엇을 보여주는지

---

2. 처음에 필요한 “데이터셋” (= 백엔드 입력 데이터)

백엔드 입장에서 데이터셋 = **학습 이미지셋이 아니라 이벤트 JSON 샘플**입니다.

2.1 최소 시작 세트 (Git에 있음 · 이것만으로 개발 시작)

경로: `docs/team-samples/2026-10-07/`

폴더/파일	내용	용도
`example_proximity_event.json`	근접 이벤트 1건	스키마 한 장 이해
`example_feature_status.json`	배너용 상태 1건	배너 API 응답 형태
`forward/events_v1.head.jsonl`	전진 근접 전환 로그 앞부분	인제스트 테스트
`forward/observations_v1.head.jsonl`	전진 프레임별 관측	배너 최신값 테스트
`reverse/...`	후진 근접	동일
`zone/...`	구역 접근/체류	동일
`lane/...`	통로 조건부 규칙	동일
`ppe5/...`	안전모(PPE)	동일

각 `*.head.jsonl`은 **앞 8줄 샘플**입니다.
형식: **한 줄 = JSON 객체 1개** (JSONL).

2.2 전체 시연 결과 (있으면 좋음 · Git에 없을 수 있음)

감지팀이 실행하면 생기는 폴더 구조 예:

```
outputs/diagnostics/<실행이름>/videoN/
  summary.json
  detections.jsonl          # 참고용(백엔드 주력 아님)
  proximity/ 또는 zone_dwell/ 또는 zone_access/ 또는 ppe/
    events_v1.jsonl         # ★ 전환 로그 (DB 저장 대상)
    observations_v1.jsonl   # ★ 프레임별 상태 (배너 대상)
```

로컬에 전체 `outputs/`가 없으면 **2.1 샘플만으로 충분**합니다.
전체 JSONL이 필요하면 감지팀에 요청하면 됩니다.

2.3 시연 영상 (선택 · UI 타임라인 맞출 때)

경로: `data/videos/` (Git에 있음)

파일	대략 기능
`1_forklift_forward.mp4`	사람-지게차 근접(전진)

파일	대략 기능
`2_forklift_back.mp4`	근접(후진)
`3_forklift_crush.mp4`	지게차-지게차 근접
`4_hazard_zone_dwell.mp4`	통로 ROI 침범·체류
`5_PPE_Helmet.mp4`	안전모
`6_Helmet_forklift.mp4`	안전모(+후드 보조 옵션)
`7_Logistics Warehouse.mp4`	종합(ROI+근접+PPE)

메타: `configs/demo-videos.json`

2.4 백엔드에 필요 없는 것

- `models/**/*.pt` (모델 가중치)
- 학습용 이미지/라벨 데이터셋
- 대용량 `outputs/` 전체(영상 mp4 포함) — 초기에는 불필요

---

3. 두 파일의 역할 (이거만 기억)

파일	의미	DB/API
`events_v1.jsonl`	상태가 **바뀐 순간** 로그	타임라인/검색
`observations_v1.jsonl`	프레임마다 **현재 상태**	상단 배너 최신값

공통 필드(이벤트):

- `schema_version`, `event_id`(재실행 시 동일 → UNIQUE로 중복 방지)
- `camera_id`, `video`, `source_sha256`
- `timestamp_seconds`
- `event_type`: `proximity` / `zone_access` / `zone_dwell` / `ppe` / `status` 등
- `severity`: `SAFE` / `WARNING` / `CRITICAL` / `null`
- `observation_status`: `confirmed` / `unconfirmed`
- `track_ids`, `evidence`(기능별 근거 JSON **원문 보존**)
- `model_version`, `config_version`, `scope`

필수 규칙:

1. `observation_status=unconfirmed` → `severity`는 **반드시 null**
2. 대상이 없다고 **SAFE를 만들지 않음**

3. 기능별로 상태를 덮어쓰지 않음 (예: 침범 CRITICAL ≠ 체류 SAFE)
4. `distance_meters`는 항상 `null` (미터 표시 금지, 화면상 거리만)
5. SAFE = 해당 기능·관측 대상 결과일 뿐, **현장 전체 안전 선언 금지**

ROI(4·7번)는 대시보드가 그려야 함:

- 이벤트 `evidence.roi_polygon_normalized` 또는 `summary.json`의 `camera_config`
- 알파: `roi_alpha_safe≈0.15`, `roi_alpha_alert≈0.2`
- SAFE=초록 / WARNING=주황 / CRITICAL=빨강

---

4. 백엔드가 만들 작업 순서 (체크리스트)

1. ☐ JSONL 줄 단위 파서
2. ☐ 테이블
   - events: 위 필드 + `evidence` JSON/TEXT
   - feature_status(또는 observations 최신): 카메라·기능별 `display_state`
3. ☐ `event_id` UNIQUE (중복 insert 무시)
4. ☐ `severity` NULL 허용
5. ☐ API
   - `GET /events` : 최신 N, 필터(`event_type`,`severity`,`camera_id`,시간)
   - `GET /events/{event_id}`
   - `GET /feature-status` : 카메라·기능별 최신 배너
6. ☐ 샘플(`docs/team-samples/2026-10-07/`)로 인제스트 → API로 다시 읽기 성공
7. ☐ 대시보드에 넘길 것: API 스펙 + 샘플 응답 2~3개 + "evidence는 JSON 그대로 전달"

배너 `feature_status.display_state`: `SAFE` / `WARNING` / `CRITICAL` / `UNKNOWN`
`global_safety_status`는 `not_evaluated`로 취급.

---

5. 대시보드에 넘겨줄 결과물 (백엔드 산출물)

백엔드가 완성되면 대시보드에 아래를 전달:

1. 이벤트 조회 API
2. feature_status(배너) API
3. 실제 응답 JSON 예시(근접/zone/ppe 각 1개)
4. ROI는 `evidence/camera_config`의 다각형으로 **보드가 직접 fill**한다는 안내
5. 문구 규칙: `docs/roi-wording-2026-10-07.md`

대시보드도 AI를 쓰면 같은 문서 + API 스펙을 첨부하면 됩니다.

---

6. 현재 한계 (화면에/문서에 같이 적을 것)

- PPE WARNING = **검토 후보** (확정 위반 증명 아님)
- 구역 ROI = **시연용** (법적 출입금지 확정 아님)
- 거리 = 화면상 거리 (`distance_meters=null`)
- 시연 적응 ≠ 독립 현장 정확도
- 모델 `.pt`는 Git에 없음(감지 실행용). 로그 연동에는 불필요

---

7. 막히면

- 문서 인덱스: `docs/team-handoff-index-2026-10-07.md`
- 메인 안내: `docs/team-integration-pack-2026-10-07.md`
- 감지팀에게: “샘플 JSONL로 인제스트는 됐는데, 전체 1~7 실행 결과 JSONL이 더 필요하다”고 요청